Assignment 1 - PS6 (Drone Path Planning using A* Search)

1. Problem Understanding
The task is to route an autonomous drone from a start cell to a destination cell in a binary terrain grid.

- `1` means the drone can fly through that cell.
- `0` means blocked region (obstacle).
- Drone can move in 8 directions: East, West, North, South, NE, NW, SE, SW.

Goal: compute an optimal path using A* search and compare two heuristics.

2. Approach Used
I modeled each valid grid cell as a node in a graph. An edge exists between two nodes if:
- the destination node is within grid bounds,
- the destination node is not blocked,
- and diagonal movement does not pass through a fully closed corner.

A* search is used with:
- `g(n)`: actual path cost from start to current node,
- `h(n)`: estimated remaining cost to destination,
- `f(n) = g(n) + h(n)`.

Priority queue (`heapq`) is used for selecting the lowest `f(n)` node efficiently.

3. Heuristics
3.1 Euclidean heuristic
$$
h(n) = \sqrt{(x_{goal}-x_n)^2 + (y_{goal}-y_n)^2}
$$

This estimates direct geometric distance to goal.

3.2 Obstacle-aware heuristic
$$
h'(n) = h(n) + \alpha \cdot O(n)
$$
where:
- `h(n)` is Euclidean distance,
- `O(n)` is count of blocked cells in 8-neighborhood around node `n`,
- `\alpha` is penalty factor (`0.35` in my implementation).

This pushes the search away from dense obstacle zones.

4. Time and Space Complexity
Let `V` be number of valid cells and `E` be number of edges.

- Time complexity: `O((V + E) log V)` due to priority queue operations.
- Space complexity: `O(V)` for `g_score`, `parent`, open/closed sets.

5. Test Results
Test Case 1
Input start: `(0,0)`, end: `(6,6)`

- Euclidean path:

`[(0, 0), (1, 0), (2, 1), (3, 1), (4, 2), (5, 3), (5, 4), (6, 5), (6, 6)]`
- Cost: `8.0`, Nodes expanded: `11`

- Obstacle-aware path:
`[(0, 0), (1, 0), (2, 1), (3, 1), (4, 2), (5, 3), (5, 4), (6, 5), (6, 6)]`
- Cost: `8.0`, Nodes expanded: `10`

Observation: both found optimal path cost, obstacle-aware heuristic expanded fewer nodes.

Test Case 2
Input start: `(0,0)`, end: `(5,5)`

- Euclidean path:
`[(0, 0), (0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 5)]`
- Cost: `6.0`, Nodes expanded: `7`

- Obstacle-aware path:
`[(0, 0), (0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 5)]`
- Cost: `6.0`, Nodes expanded: `8`

Observation: both heuristics found the same optimal path here.

Note: A* can return one among multiple equally optimal paths depending on tie-breaking in priority queue and neighbor expansion order.

6. Alternate Modeling (Required)
Alternate model: weighted graph with risk score per cell.

Instead of binary free/blocked map only, assign each free cell a risk value (for example based on wind/turbulence/no-fly proximity). Then define edge cost as:
\[
Cost = movement_cost + \lambda \cdot risk
\]

Performance implications
- Pros:
- Better real-world modeling of “safe” navigation.
- Naturally balances shortest path and safest path.
- Cons:
- More preprocessing required.
- Slightly higher constant overhead in evaluating each transition.

7. Error Handling Included
- Empty terrain check.
- Out-of-bound start/end check.
- Start/end blocked check.
- Input format validation for terrain, start, end.
- Graceful message when no path exists.

8. Conclusion
A* with Euclidean heuristic gives a strong baseline for shortest path planning. The obstacle-aware heuristic improves safety bias and can reduce exploration in cluttered regions. The implementation is modular, testable, and compatible with file-based evaluation.